

E-Commerce Sales Analytics and Product Recommendation using Hadoop, Hive and Spark

BCSE402L – Big Data Analytics (TH) | DA-1: Topic Selection & Planning

Team Members: [Name 1 – Reg. No., Team Lead], [Name 2 – Reg. No.], [Name 3 – Reg. No.], [Name 4 – Reg. No.]

Submission Date: 4 August 2026

1. Problem Statement

Domain: E-Commerce Analytics (Brazilian marketplace transaction data – Olist).

Problem (one line): To build a batch Big Data analytics pipeline over multi-CSV e-commerce transaction data that uncovers sales trends and customer behaviour and generates personalised product recommendations at a scale where single-machine tools fail.

Why this problem is meaningful:

- Recommendations drive a large share of e-commerce revenue (industry estimates go up to 35%); small marketplaces lack affordable analytics stacks.
- Transaction data arrives as multiple large relational CSVs (orders, items, payments, reviews) whose joins and aggregations outgrow in-memory tools as volume grows.
- Insights such as top categories, delivery delays, payment preferences and demand seasonality directly support inventory and marketing decisions.
- It is a classic volume + variety Big Data problem, ideal for the Hadoop ecosystem tools taught in BCSE402L.

2. Course Relevance

The project maps directly onto four modules of the course syllabus:

Course Module	Concept	Use in Project
Module 2 – Hadoop, HDFS, Hive, Pig	HDFS storage, HQL, partitioning, Pig Latin	HDFS data lake for all CSVs; Hive external tables partitioned by purchase month; Pig scripts for cleaning.
Module 3 – MapReduce	Map/Reduce, key-value pairs, combiners, partitioners	Custom MR jobs: revenue per product category, orders per state, payment-type distribution (combiners for local aggregation).
Module 4 – NoSQL (MongoDB)	Document store	Serving layer storing per-user recommendation lists for fast lookup.
Module 5 – Spark	DataFrames, Spark SQL, MLlib, ETL	Multi-table joins and feature preparation; ALS collaborative-filtering recommender; reporting DataFrames.

3. Basic Concepts

3.1 Tools

- HDFS: splits files into 128 MB replicated blocks; gives horizontal scalability and fault tolerance.
- MapReduce: mappers emit (key, value) pairs, e.g. (category, revenue); shuffle groups by key; reducers aggregate; combiners cut network traffic.
- Apache Hive: SQL-like warehouse layer (HQL) over HDFS with metastore, partitioning and bucketing.
- Apache Spark + MLlib: in-memory DataFrames for joins/ETL; ALS matrix factorisation for recommendations.
- MongoDB: stores final recommendation documents of the form {user_id, [product_ids]}.

3.2 Algorithm – ALS Collaborative Filtering

- A ratings matrix is built from purchase counts and review scores (users $\times$ products).
- ALS alternates between fixing user and item factor matrices to minimise regularised least-squares error — naturally parallel and supported by Spark MLlib.
- Evaluation: RMSE on held-out ratings and top-N hit-rate; popularity-based fallback for cold-start users.

3.3 Workflow Diagram

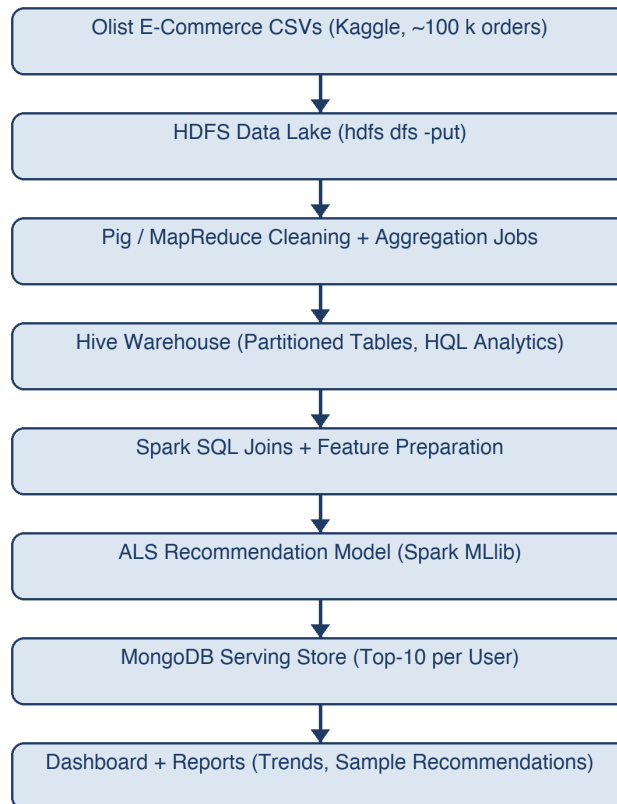


Fig. 1: End-to-end batch analytics and recommendation workflow.

4. Approach & Roles

4.1 Methodology

1. Data collection – Brazilian E-Commerce Public Dataset (Olist, Kaggle): 9 CSVs, ~100 k orders (2016–2018), ≈ 120 MB; optional synthetic scale-up to the GB range.
2. Environment setup – Hadoop pseudo-distributed (HDFS + YARN), Hive, Spark, MongoDB.
3. Ingestion & cleaning – hdfs dfs -put; Pig/MR cleaning: null handling, type casting, de-duplication, timestamp parsing.
4. MapReduce analytics – revenue by category, orders by state, payment distribution, monthly order counts.
5. Hive warehouse – external/partitioned tables; HQL queries for monthly GMV trend, top categories, delivery-delay analysis, repeat-customer rate.
6. Spark ETL + recommendation – join orders/items/payments/reviews; build ratings; train ALS; generate top-10 recommendations per user.
7. Serving & visualisation – recommendations written to MongoDB; Matplotlib/Streamlit dashboard for trends and sample recommendations.
8. Evaluation & documentation – RMSE/hit-rate; DA-2 and DA-3 deliverables.

4.2 Tentative Timeline

Phase	Work	Duration
Phase 1	Setup (Hadoop/Hive/Spark/MongoDB), dataset download	4 – 10 Aug
Phase 2	HDFS ingestion + Pig/MR cleaning	11 – 24 Aug
Phase 3	MapReduce analytics jobs + Hive warehouse	25 Aug – 7 Sep
Phase 4	Spark ETL + ALS recommender (DA-2: 19 Sep)	8 – 19 Sep
Phase 5	MongoDB serving + dashboard + tuning	20 Sep – 12 Oct
Phase 6	Final report + presentation (DA-3: 20 Oct)	13 – 20 Oct

4.3 Team Roles

Member	Responsibility
--------	----------------

Member 1 – [Name] (Lead)	Hadoop/Hive cluster setup, overall architecture, integration
Member 2 – [Name]	Data ingestion and cleaning (Pig / MapReduce)
Member 3 – [Name]	MapReduce + HQL analytics, results documentation
Member 4 – [Name]	Spark ALS recommender, MongoDB serving layer, dashboard

5. Feasibility

5.1 Dataset Sources

- Brazilian E-Commerce Public Dataset by Olist (Kaggle, public licence): 9 relational CSVs, ~100 k orders, ≈ 120 MB – identified and downloaded.
- Growth path if more volume is needed: synthetic order generator or a larger public clickstream dataset (e.g. REES46).

5.2 Evidence of Setup (Initial Progress)

- Hadoop single-node (pseudo-distributed) cluster running; hdfs dfs -ls verified.
- Hive metastore initialised; sample HQL queries executed on the dataset.
- Spark + PySpark verified; MongoDB Community Server running.
- Dataset CSVs inspected (schemas, row counts); Git repository initialised.

5.3 Expected Challenges & Mitigation

Challenge	Mitigation
Single-node hardware limits	Pseudo-distributed config; develop on samples, full run for final results
Skewed keys (a few big categories)	Combiners and a custom partitioner
Complex joins in raw MapReduce	Do joins in Hive/Spark; keep MR for aggregations only
Cold-start users in ALS	Popularity-based fallback recommendations
Hadoop/Hive/Spark version compatibility	Pin compatible versions; document the setup steps

The dataset is public, the full stack is open-source, and the cluster is already running on a single machine — the project is feasible within the semester timeline.